

Algorithm III – Week 2 Lesson Plan

Dynamic Arrays: Resizing, Searching, Insertion, Deletion, and Boundary Testing

Lesson Objectives

Knowledge

- Distinguish an array's logical size (elements currently stored) from its capacity (allocated element slots).
- Explain why resizing requires preserving valid elements, updating capacity, and protecting the original allocation when realloc fails.
- Describe the state changes required for linear search, insertion, deletion, expansion, and optional shrinking.
- Explain the boundary conditions for empty, full, first-position, last-position, and invalid-position operations.
- Compare the immediate cost of search, insertion, deletion, and expansion in a contiguous dynamic array.

Skills

- Trace size, capacity, indexes, and element movement after each operation.
- Use realloc safely through a temporary pointer and update the owner only after success.
- Implement modular C functions for search, insert, remove, expand, and shrink.
- Build a menu-driven dynamic integer collection with clear operation results.
- Design and run normal, boundary, invalid-input, allocation-failure, and cleanup tests.

Attitude

- Treat array metadata as part of the data structure's correctness.
- Prefer explicit invariants and test evidence over code that only appears to work.
- Preserve data and ownership when an operation cannot be completed.

Lesson Structure (3 Hours)

Part 1: Week 1 Bridge and Diagnostic Trace (15 mins)

Begin with a valid Week 1 allocation and ask what is missing if the program later needs a sixth element.

```
int size = 4;
int capacity = 4;
int *data = malloc(capacity * sizeof *data);
/* data contains: 10, 20, 30, 40 */
```

- Which variable describes the number of meaningful elements?
- Which variable describes the number of allocated slots?

- Is data[4] valid before expansion?
- What must remain true if expansion fails?

Key bridge: Week 1 controlled one allocation lifetime. Week 2 changes the allocation while preserving its values, ownership, and valid bounds.

Part 2: Logical Size, Capacity, and Invariants (20 mins)

Term	Meaning	Example
size	Number of meaningful elements currently stored	3
capacity	Number of allocated element slots	5
valid element indexes	Indexes that contain collection values	0 through size – 1
allocated indexes	Indexes inside the block, including unused slots	0 through capacity – 1

- Invariant 1: $0 \leq \text{size} \leq \text{capacity}$.
- Invariant 2: data is NULL when capacity is 0; otherwise it identifies a live allocation.
- Invariant 3: only indexes 0 through size – 1 are meaningful collection elements.
- Invariant 4: the owner releases the live block exactly once.

Emphasize that an allocated slot is not automatically a valid collection element. Reading data[size] is logically invalid even when size < capacity.

Part 3: Safe Expansion with realloc (25 mins)

```
int newCapacity = capacity * 2;
int *temp = realloc(data, newCapacity * sizeof *data);
if (temp == NULL) {
    /* data is still valid; size and capacity stay unchanged */
} else {
    data = temp;
    capacity = newCapacity;
}
```

- realloc may keep the block at the same address or move it.
- On success, the old pointer must no longer be used independently.
- On failure, realloc returns NULL and the original block remains allocated.
- Assigning realloc directly to data can lose the only pointer to the original block on failure.
- Capacity changes only after successful resizing.

Use doubling as the default growth policy for this lesson. It reduces how often copying is required compared with increasing capacity by one every time.

Part 4: Linear Search and Result Design (15 mins)

```
int findIndex(const int *data, int size, int target) {
    for (int i = 0; i < size; i++) {
        if (data[i] == target) return i;
    }
    return -1;
}
```

- Search only the logical elements: $i < \text{size}$, not $i < \text{capacity}$.
- Return the first matching index; return -1 when the target is absent.
- For an empty collection, the loop executes zero times and returns -1 .

Part 5: Insertion and Right Shift (25 mins)

```
/* position is valid when 0 <= position <= size */
if (size == capacity) {
    /* expand first; stop without change if expansion fails */
}
for (int i = size; i > position; i--) {
    data[i] = data[i - 1];
}
data[position] = value;
size++;
```

- Insertion at position size appends and moves no existing element.
- Shift from right to left to avoid overwriting values that still need to be copied.
- The empty collection permits insertion only at position 0.
- Increment size only after the value has been stored successfully.

State	size	capacity	Meaningful values
Before	3	5	10, 20, 30
Shift data[2] → data[3]	3	5	10, 20, 30, 30
Shift data[1] → data[2]	3	5	10, 20, 20, 30
Store 15 at index 1	3	5	10, 15, 20, 30
Commit size++	4	5	10, 15, 20, 30

Part 6: Deletion and Left Shift (20 mins)

```
/* position is valid when 0 <= position < size */
int removed = data[position];
for (int i = position; i < size - 1; i++) {
    data[i] = data[i + 1];
}
size--;
```

- Deletion requires an existing element, so position must be less than size.
- Shift from left to right because each later element fills the preceding gap.

- Deleting the last element performs no shift.
- The old value beyond the new logical end may remain in memory but is no longer part of the collection.

Part 7: Shrinking Policy and Safe Resize Function (15 mins)

Shrinking is optional and must never make capacity smaller than size. Use a conservative policy to avoid repeated grow/shrink cycles.

```
if (capacity > MIN_CAPACITY && size <= capacity / 4) {
    int newCapacity = capacity / 2;
    if (newCapacity < MIN_CAPACITY) newCapacity = MIN_CAPACITY;
    int *temp = realloc(data, newCapacity * sizeof *data);
    if (temp != NULL) {
        data = temp;
        capacity = newCapacity;
    }
    /* If shrinking fails, keep the larger valid block. */
}
```

Part 8: Operation Costs and Design Trade-offs (15 mins)

Operation	Typical work	Basic time cost
Index access	Directly access data[i]	Constant
Linear search	Inspect up to size elements	Linear
Append with free capacity	Store one value	Constant
Insert/delete near beginning	Shift many elements	Linear
Expansion	Allocator may move/copy size elements	Linear for that expansion

Keep the discussion qualitative in Week 2. Formal asymptotic notation and time–space analysis are consolidated in Week 5.

Part 9: Integrated Trace, Boundary Testing, and Exit Check (30 mins)

Trace this operation sequence using a state table: initialize capacity 2; append 10; append 20; insert 15 at position 1; search for 20; delete position 0; append 30.

Step	size	capacity	Meaningful values	Safety/cost note
Initial	0	2	—	Live allocation; empty
Append 10	1	2	10	No resize
Append 20	2	2	10, 20	Now full

Step	size	capacity	Meaningful values	Safety/cost note
Insert 15 at 1	3	4	10, 15, 20	Expand, then shift right
Search 20	3	4	10, 15, 20	Return index 2
Delete 0	2	4	15, 20	Shift left
Append 30	3	4	15, 20, 30	No resize

Student Explanation Sequence

1. What are size and capacity before the operation?
2. Which positions are valid for this operation?
3. Is expansion required? If so, what new capacity is requested?
4. Which direction must elements move, and why?
5. When is the metadata update committed?
6. If allocation fails, which state must remain unchanged?
7. What normal and boundary tests demonstrate correctness?

Evaluation Questions

- Why can size be smaller than capacity but never larger?
- Why should search stop at size rather than capacity?
- Why is a temporary pointer required with realloc?
- What remains valid when realloc returns NULL?
- Why must insertion shift from right to left?
- Why must deletion shift from left to right?
- What is the valid insertion-position range for size n ?
- What is the valid deletion-position range for size n ?
- Why does inserting at position size mean append?
- Why is the old last slot not a valid element after size decreases?
- When is shrinking useful, and why should it be conservative?
- Which tests cover empty, full, first, last, invalid, and allocation-failure cases?

Teacher Checklist

- Student maintains $0 \leq \text{size} \leq \text{capacity}$.
- Student distinguishes meaningful elements from unused allocated slots.
- Student protects the original pointer during realloc.
- Student validates positions before moving elements.
- Student shifts in the correct direction and updates size once.

- Student preserves the collection when an operation fails.
- Student explains and demonstrates boundary tests.
- Student releases the final live allocation exactly once.

Teaching Notes and Source Alignment

- Keep implementation in C with malloc, realloc, free, and NULL.
- Use a small integer collection so every state change can be drawn.
- Do not introduce structs for the collection yet; structs are a Week 3 topic.
- Tony Gaddis, Starting Out with C++: use the array searching and vector growth/operation discussions as conceptual references; translate them into explicit C memory management.
- The Week 2 mission should prepare students to implement the practical exercises rather than reveal a complete final program.

Key Teaching Philosophy

“A dynamic array is correct only when its values, size, capacity, ownership, and failure behavior remain consistent after every operation.”